DATA - 260
HomeWork - 8
Name: Karthik Pragada
SJSU ID: 018768360

GitHub repository: https://github.com/prags8585/data260-8360 (link checked 2026-11-01: HTTP 200)

Field	Value
SID4	8360
PORT_BASE	8260 (8000 + 8360 mod 900)
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0 (8360 mod 8) - Campus Course Catalogue and Enrolment; primary entity: Course
HARDWARE	MacBook Air, Apple M4, 16 GB RAM
Local Model Used	qwen3:8b via Ollama (Part 2 agents; Parts 1 and 3 use no model)
Tagged commit hash	to be filled in after the commit is tagged hw8

Homework 8 extends the same repository again. The domain logic of Part 1 is split into a client and a worker that talk only through Kafka (a request and a response topic, a FastAPI wrapper in front). Part 2 runs a Planner, a Writer and a Reviewer as three separate processes that exchange messages through four topics. Part 3 breaks the system on purpose (duplicate delivery, a killed consumer, a poison message, ordering across two partitions) and shows each problem before and after its fix. Broker: Apache Kafka 3.8.0 (KRaft, one node) in Docker, Python client confluent-kafka. Terminal figures are captured from real commands on this machine and show my name in the title bar; raw data for every number is in `reports/hw08/raw/`. Container log lines are in UTC (trailing Z).

Part 1 - Backend Service Request/Response over Kafka

I. Operations and entity

The primary entity is **Course**, so the operations are `CREATE_COURSE` and `GET_COURSE` and the identifier is `course_id` (a UUID generated by the worker). Required fields (same rules as the HW5 schema): `courseTitle` (non-empty text), `courseCode` (like DATA-260), `seatsAvailable` (integer 0-1000) and `instructorId` (positive integer); `department` and `description` are optional text. Missing, empty, mistyped or unknown fields are rejected with a clear error. The worker keeps the entities in a plain Python dictionary, as required.

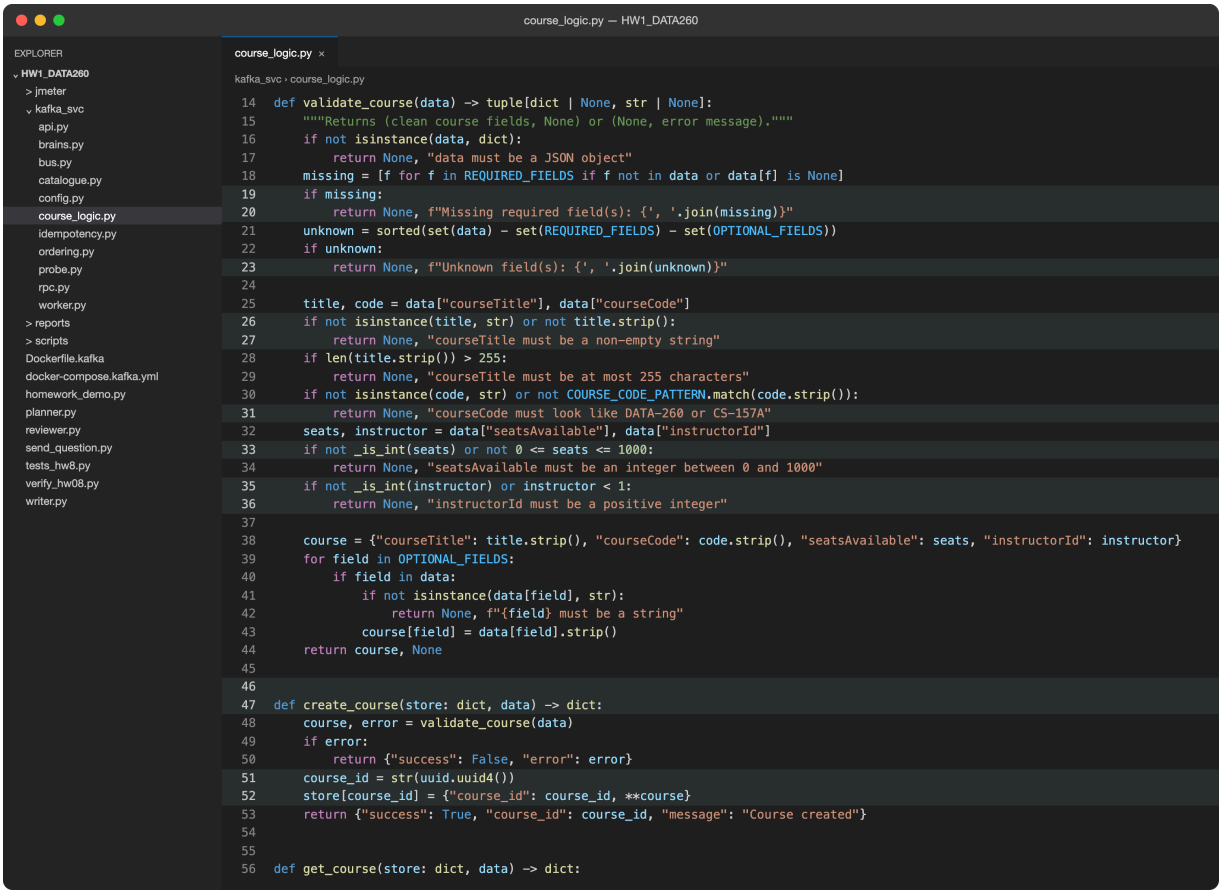


Figure 1.1 - kafka_svc/course_logic.py: field validation and CREATE_COURSE (UUID id).

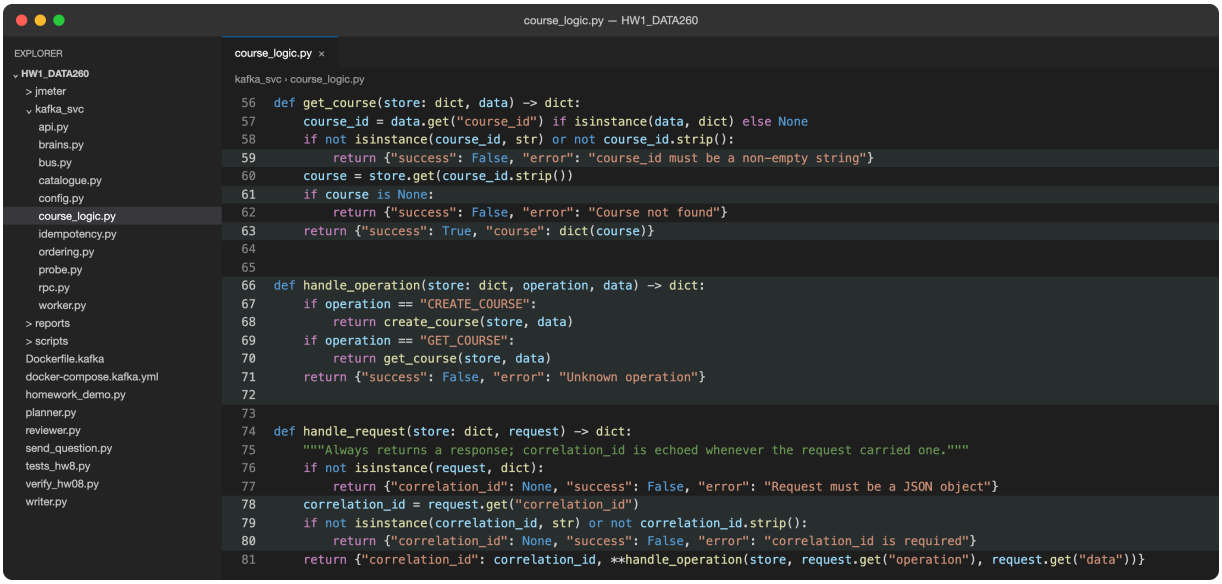


Figure 1.2 - kafka_svc/course_logic.py: GET_COURSE, unknown-operation handling and the response envelope that always echoes correlation_id.

II. Kafka topics and message schemas

Topic	Written by	Read by	Partitions
`s8360_entity_requests`	client / FastAPI wrapper	worker (group s8360_entity_worker)	1
`s8360_entity_responses`	worker	client / FastAPI wrapper	1

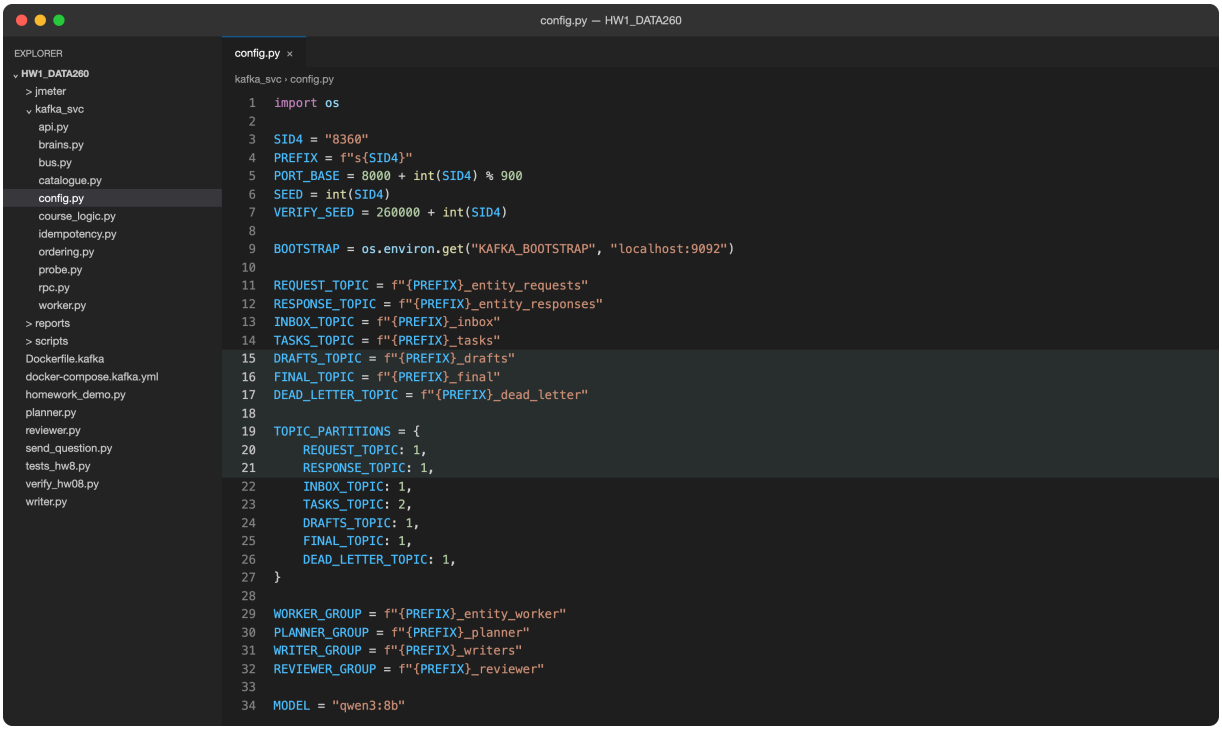


Figure 1.3 - kafka_svc/config.py: the personal values and every topic name (PREFIX s8360).

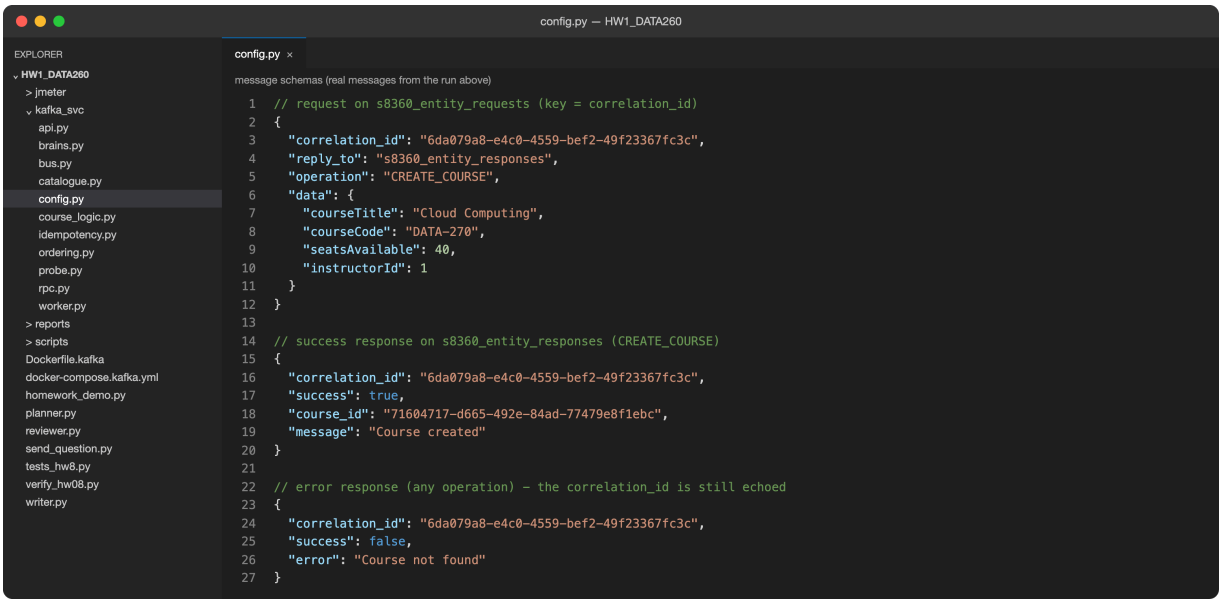


Figure 1.4 - The request and the two response schemas, taken from a real run (the correlation_id is created once by the client and never changed).

#	Component	What happens to correlation_id
1	client (homework_demo.py / KafkaRpcClient, or FastAPI via X-Correlation-ID)	created once before the request is sent; remembered in a pending-reply table
2	request topic	the request message carries correlation_id, reply_to, operation, data (message key = correlation_id)
3	worker	reads the operation, validates, builds the response with the same correlation_id (also for errors)
4	response topic	the worker sends the response to the topic named by reply_to
5	client / FastAPI	a reply thread matches the correlation_id to the waiting caller; FastAPI returns it in the body and the X-Correlation-ID header

III. Code: Kafka client, worker, API wrapper and demonstration

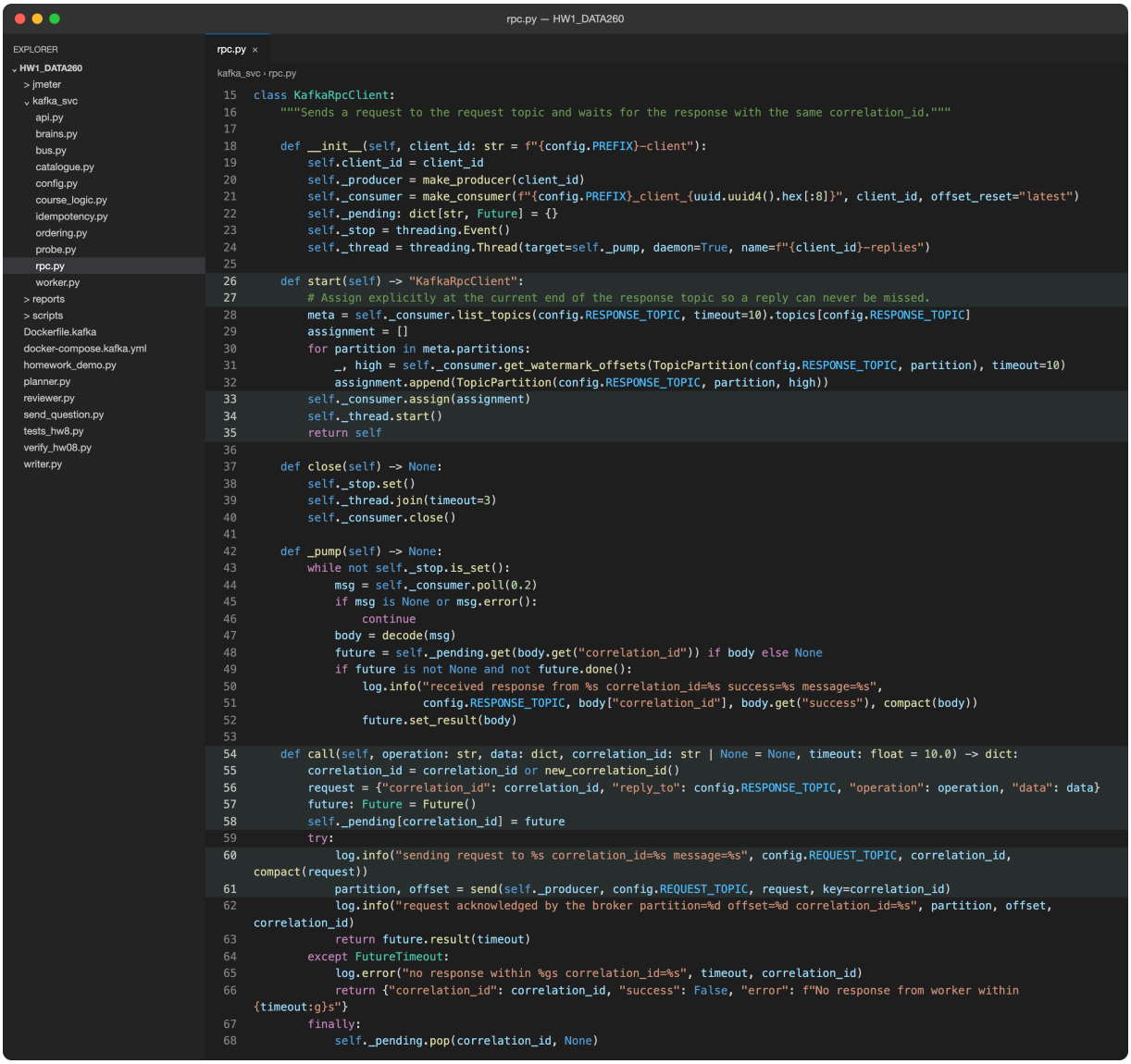


Figure 1.5 - kafka_svc/rpc.py: KafkaRpcClient - send the request, wait for the reply with the same correlation_id.

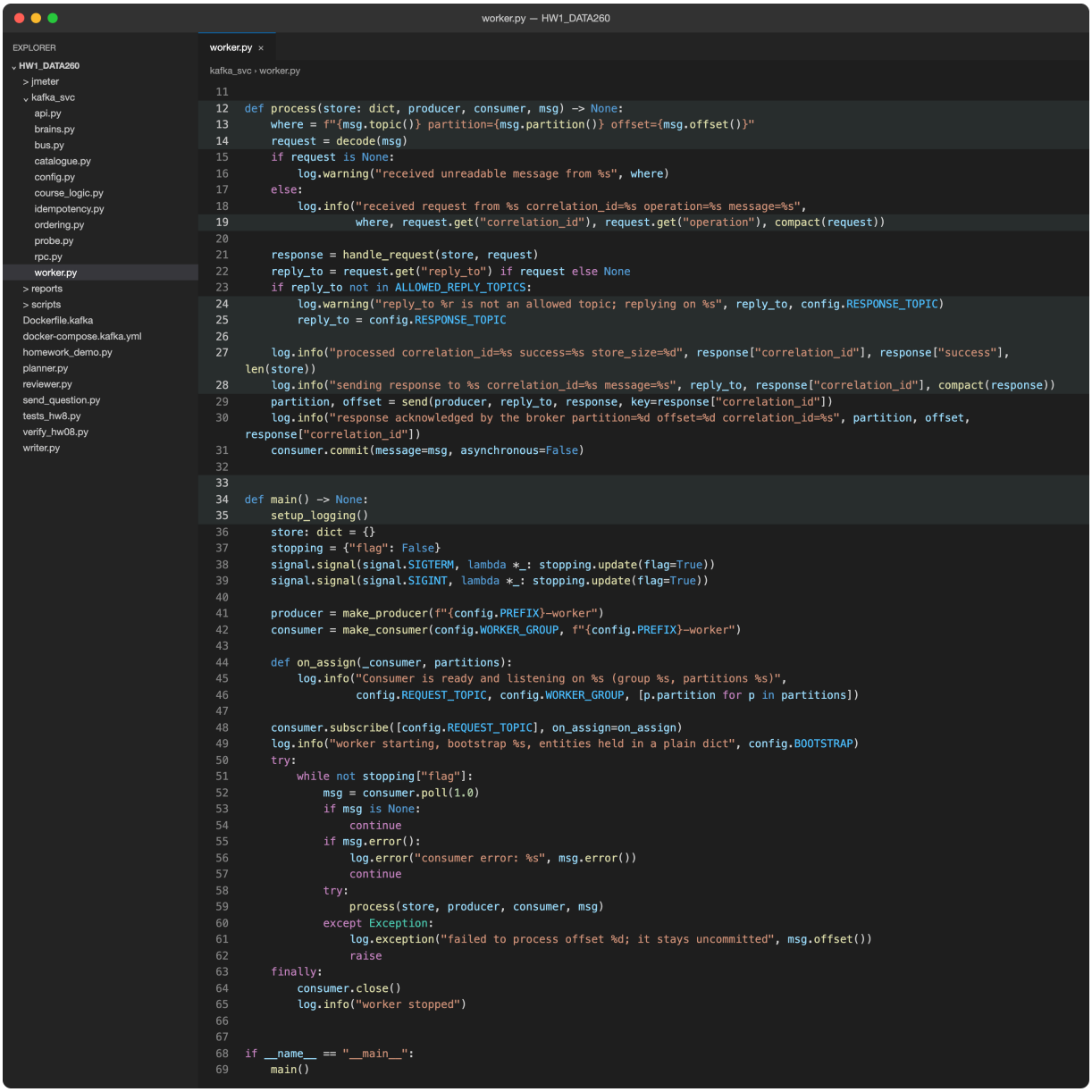


Figure 1.6 - kafka_svc/worker.py: consume, validate/process, reply to reply_to, commit the offset after the reply is acknowledged.

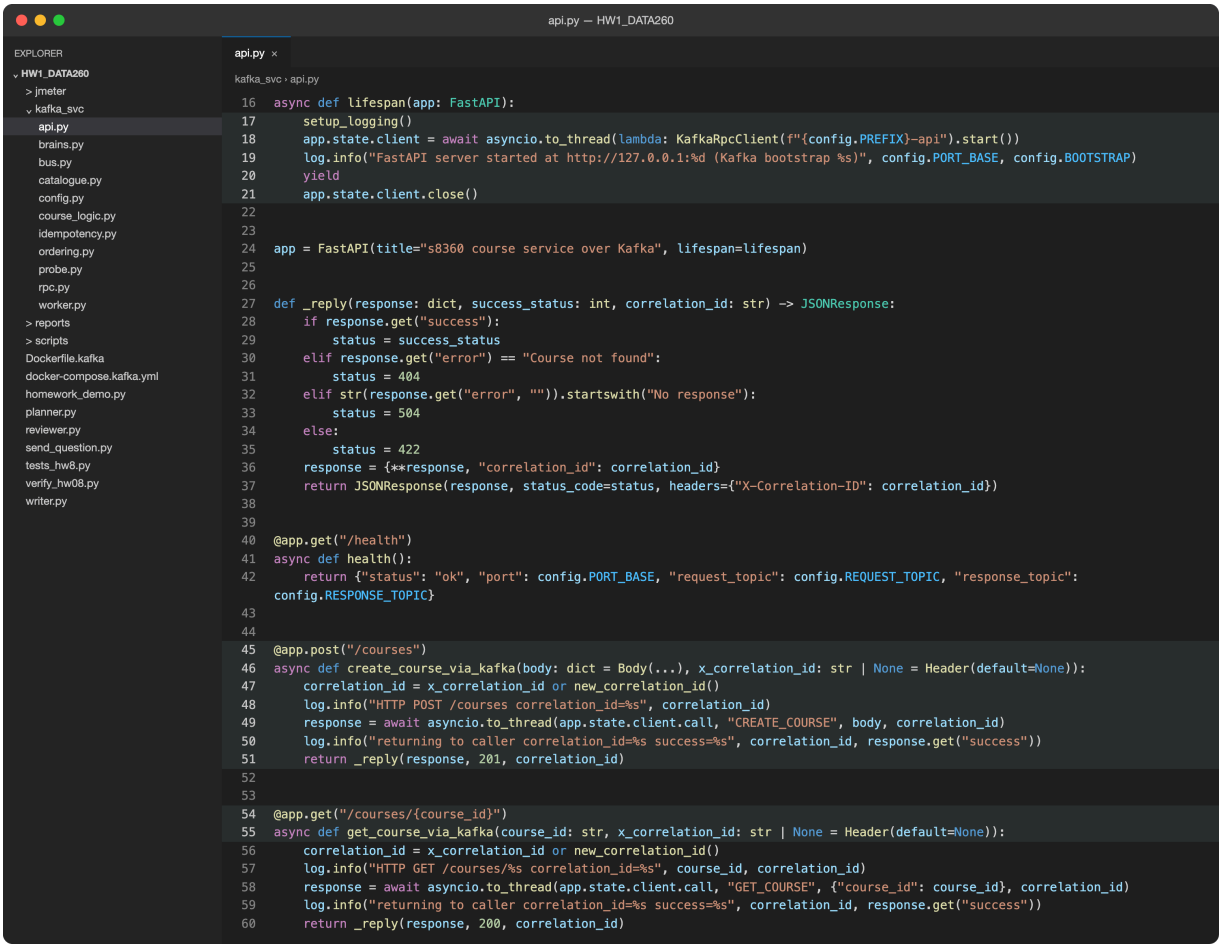


Figure 1.7 - kafka_svc/api.py: FastAPI wrapper (POST /courses, GET /courses/{id}); the log line at startup is the one the assignment asks for.

homework_demo.py — HW1_DATA260

EXPLORER

- HW1_DATA260
  - > jmeter
  - > kafka_svc
  - > reports
  - > scripts
  - Dockerfile.kafka
  - docker-compose.kafka.yml
  - homework_demo.py
  - planner.py
  - reviewer.py
  - send_question.py
  - tests_hw8.py
  - verify_hw08.py
  - writer.py

homework_demo.py

```python
def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--via", choices=["kafka", "api"], default="kafka", help="send from the Kafka client directly or through the FastAPI wrapper")
    args = parser.parse_args()

    setup_logging()
    collector = Collector()
    collector.setFormatter(logging.getLogger().handlers[0].formatter)
    logging.getLogger().addHandler(collector)
    since = (datetime.now(timezone.utc) - timedelta(seconds=1)).strftime("%Y-%m-%dT%H:%M:%SZ")

    client = KafkaRpcClient(f"{config.PREFIX}-demo").start() if args.via == "kafka" else None
    send = (lambda op, data, cid: call_kafka(client, op, data, cid)) if client else call_api

    correlation_id = new_correlation_id()
    log.info("step 1: created correlation_id=%s (route: %s)", correlation_id, "Kafka client" if client else "FastAPI -> Kafka")
    log.info("step 2: sending CREATE_COURSE to %s (correlation_id=%s)", config.REQUEST_TOPIC, correlation_id)
    created = send("CREATE_COURSE", COURSE, correlation_id)
    course_id = created.get("course_id")
    log.info("step 7: extracted course_id=%s from the CREATE_COURSE response (correlation_id=%s success=%s)", course_id, correlation_id, created.get("success"))
    if not created.get("success"):
        log.error("CREATE_COURSE failed: %s", created)
        return 1

    log.info("step 8: sending GET_COURSE for course_id=%s (correlation_id=%s)", course_id, correlation_id)
    fetched = send("GET_COURSE", {"course_id": course_id}, correlation_id)
    log.info("step 10: retrieved entity correlation_id=%s course=%s", correlation_id, compact(fetched.get("course")))
    preserved = created.get("correlation_id") == correlation_id and fetched.get("correlation_id") == correlation_id
    log.info("step 10: correlation_id preserved end to end: %s (sent %s, CREATE reply %s, GET reply %s)",
             preserved, correlation_id, created.get("correlation_id"), fetched.get("correlation_id"))
    if client:
        client.close()

    time.sleep(0.5)
    names = ["s8360-api", "s8360-worker"] if not client else ["s8360-worker"]
    print("\n=== merged timeline (demo client + container logs, ordered by timestamp, filtered to this correlation_id) ===")
    merged = [(line.split(" ", 1)[0], line) for line in collector.lines if correlation_id in line]
    merged += [(line.split("] ", 1)[1].split(" ", 1)[0], line) for line in container_lines(names, since, correlation_id)]
    for _, line in sorted(merged, key=lambda pair: pair[0]):
        print(line if line.startswith("[") else f"[demo] {line}")
    return 0 if preserved and fetched.get("success") and fetched["course"]["course_id"] == course_id else 1


if __name__ == "__main__":
    sys.exit(main())
```

Figure 1.8 - homework_demo.py: the 10-step demonstration (steps 1, 2, 7, 8 and 10 are highlighted).

IV. Required screenshots and evidence

1. Containers. `docker ps` shows the Kafka, worker and FastAPI containers; the worker log shows "Consumer is ready and listening..." and the FastAPI log shows the server started at http://127.0.0.1:8260. The four-container stack was started from scratch (`docker compose -f docker-compose.kafka.yml up -d --build`) and a one-shot job created the topics.

Karthik Pragada — prags@Karthiks-MacBook-Air — -zsh

```
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}' | grep -E 'NAMES|s8360-(kafka|worker|api)'
NAMES                       IMAGE                       STATUS                   PORTS
s8360-api                   s8360-hw8-kafka-svc         Up 10 seconds (healthy)   0.0.0.0:8260->8260/tcp, [::]:8260->8260/tcp
s8360-worker                s8360-hw8-kafka-svc         Up 10 seconds
s8360-kafka                 apache/kafka:3.8.0          Up 23 seconds (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % docker logs s8360-worker 2>&1 | grep -E 'worker starting|Consumer is ready'
2026-11-01T23:13:28.143Z worker INFO worker starting, bootstrap kafka:19092, entities held in a plain dict
2026-11-01T23:13:31.185Z worker INFO Consumer is ready and listening on s8360_entity_requests (group s8360_entity_worker, partitions [0])
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % docker logs s8360-api 2>&1 | grep -E 'FastAPI server started|Uvicorn running'
2026-11-01T23:13:28.434Z api INFO FastAPI server started at http://127.0.0.1:8260 (Kafka bootstrap kafka:19092)
INFO:     Uvicorn running on http://0.0.0.0:8260 (Press CTRL+C to quit)
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Figure 1.9 - docker ps with s8360-kafka, s8360-worker and s8360-api, plus the worker and FastAPI startup log lines.

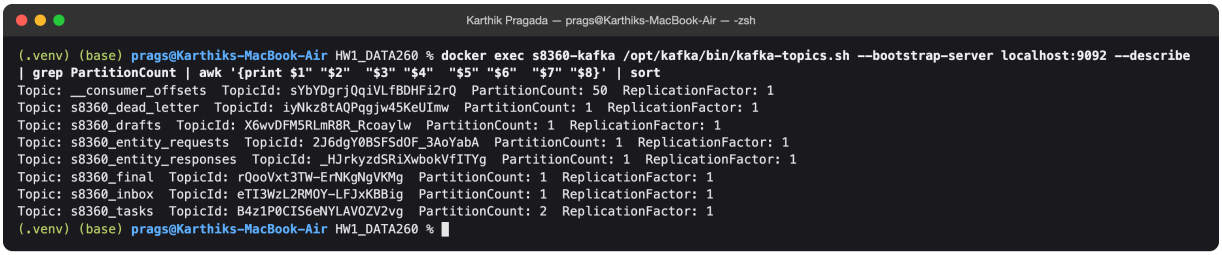


Figure 1.10 - kafka-topics.sh --describe: the topics and their partition counts (s8360_tasks has 2).

2. CREATE_COURSE and GET_COURSE execution. homework_demo.py sends both operations and prints operation, entity data, generated course_id, success status and correlation_id.

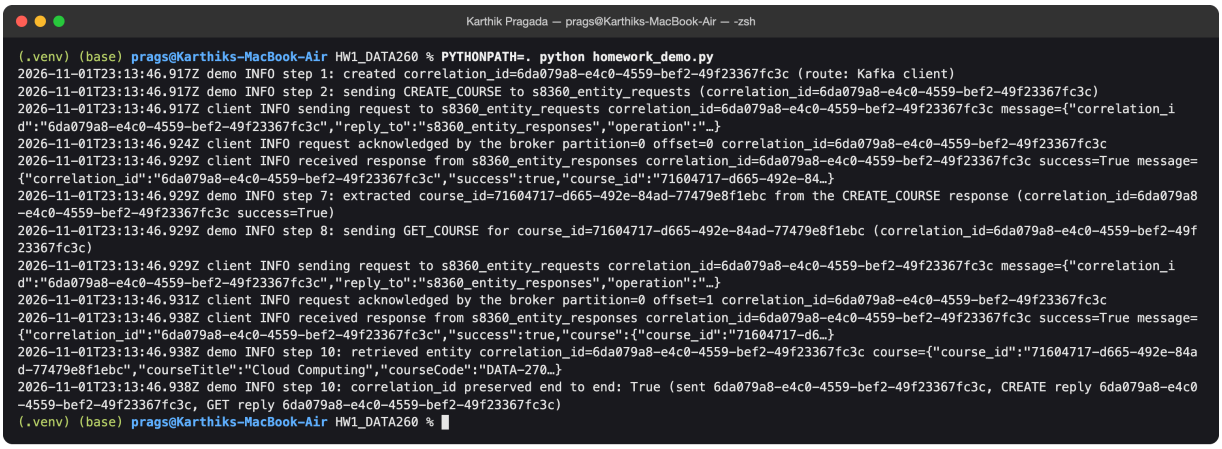


Figure 1.11 - homework_demo.py console output (client side).

3-4. Kafka message flow and full sequence. The second half of the same run merges the client log and the worker's container log by timestamp, filtered to this correlation_id: send, receive, process, reply and the final client-side response, in order, for CREATE_COURSE and then GET_COURSE.

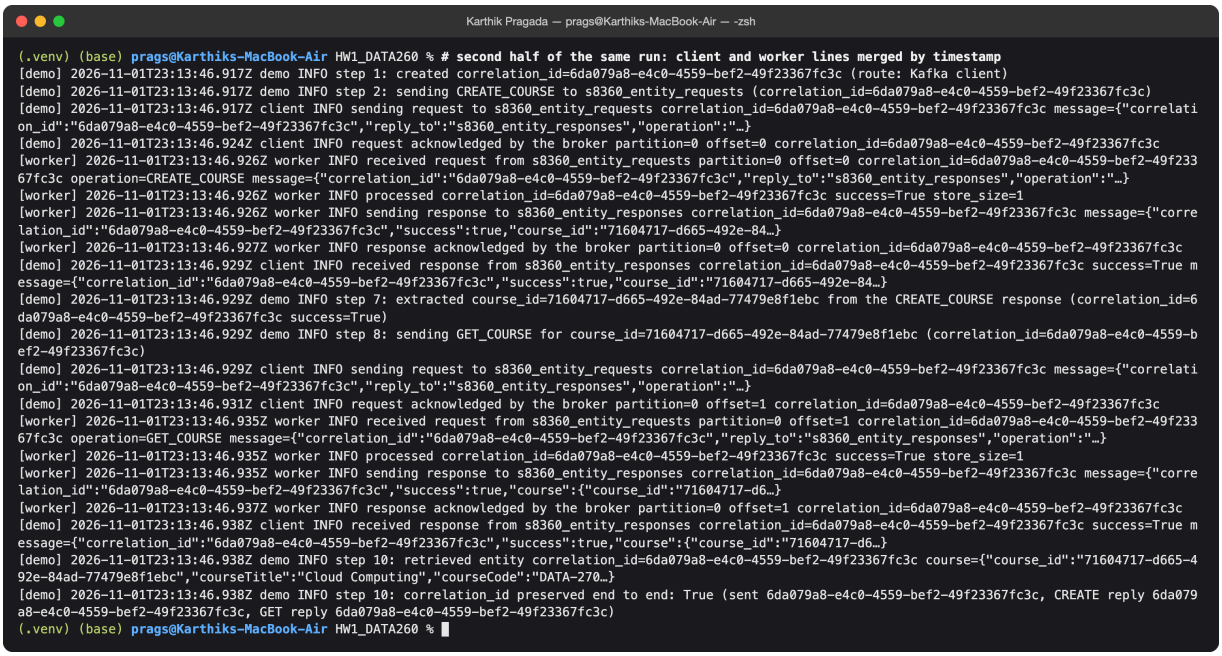


Figure 1.12 - CREATE_COURSE -> GET_COURSE: send, receive, process, reply, client response (long JSON payloads shortened with ...; full lines in RUN_LOG.txt).

The same flow through the FastAPI wrapper (API call). The API container logs its own send/receive lines, the worker logs the processing and the demo prints the HTTP status codes.

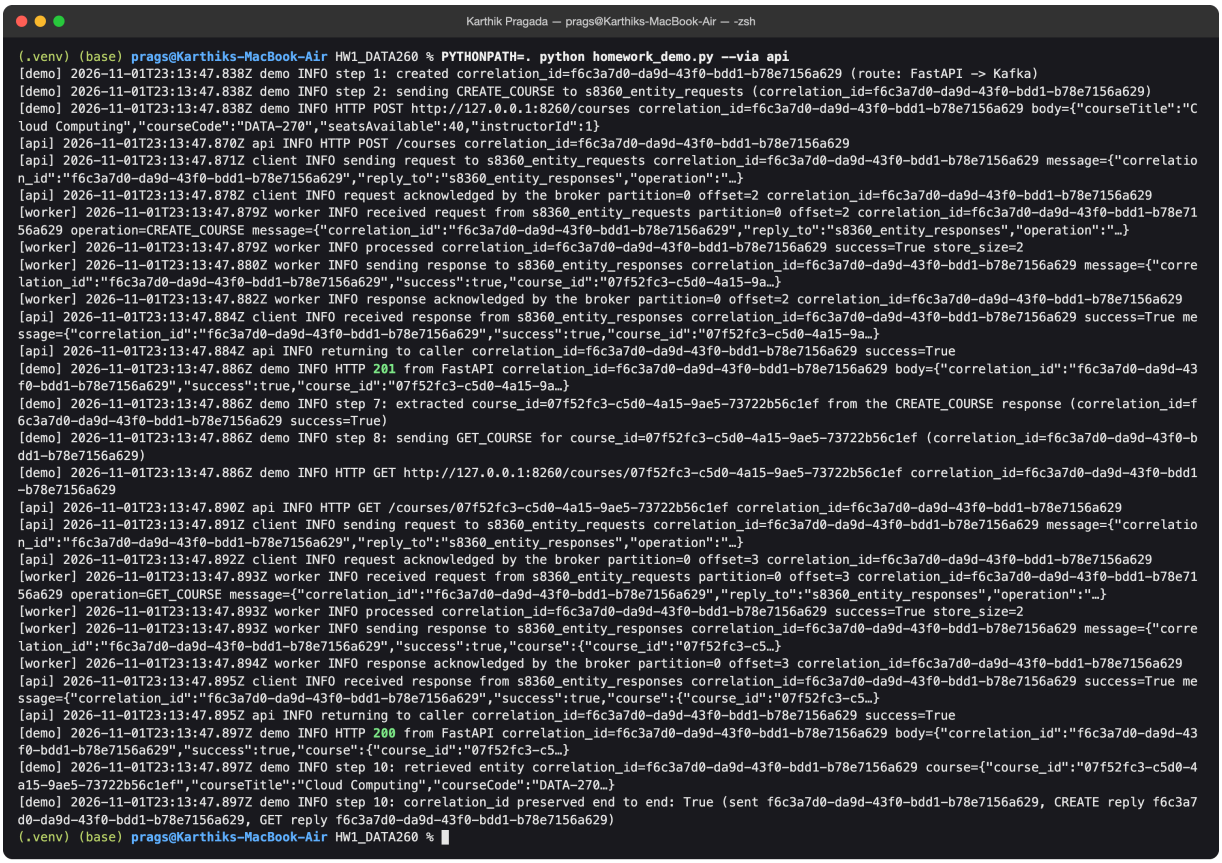


Figure 1.13 - homework_demo.py --via api: HTTP 201 / 200 from FastAPI, with the API and worker log lines in between.

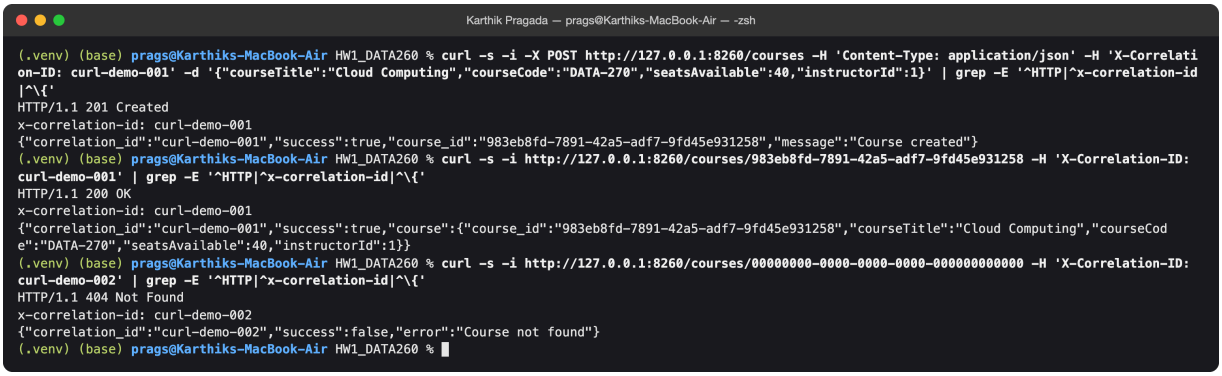


Figure 1.14 - curl against the API: the X-Correlation-ID sent by the caller is returned unchanged (also for the 404).

V. Error handling

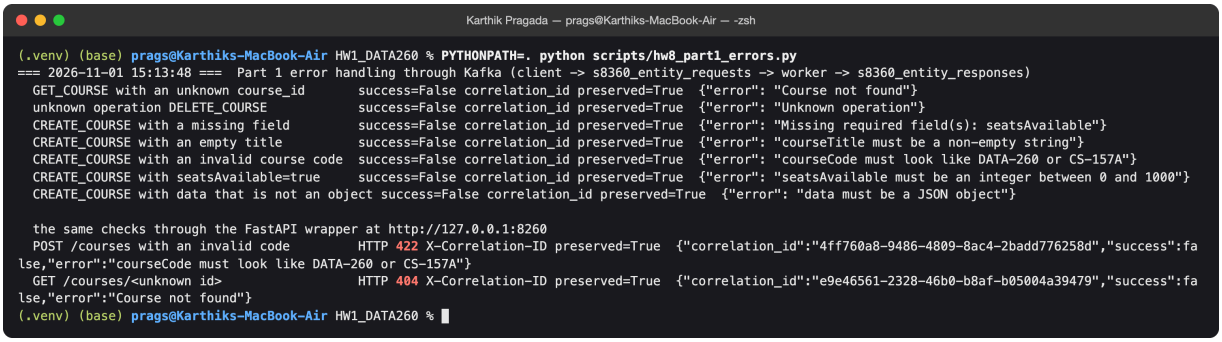


Figure 1.15 - Unknown id, unknown operation and invalid fields: success=false, a clear error and the same correlation_id (through Kafka and through FastAPI).

Part 2 - Three-Agent Kafka Pipeline

I. Topics and workflow

Sender	Reads topic	Sends message to
send_question.py (me)	-	`s8360_inbox` (a question)
Planner	`s8360_inbox`	`s8360_tasks` (a plan)
Writer	`s8360_tasks`	`s8360_drafts` (a draft answer)
Reviewer	`s8360_drafts`	`s8360_final` (an approval)

Planner and Writer use qwen3:8b (temperature 0, seed 8360). The Writer first runs the HW5 catalogue tools (`course_stats` , `search_courses`) over an in-memory snapshot of six courses and gives the model only what the tools returned. The Reviewer is deterministic: it checks the length, that every number in the draft appears in the facts, that every course code exists, and that a "most seats" claim names the right department and course. No agent calls another agent; the only connection is the topics. The four topics were created with the CLI commands from the assignment (tasks has 2 partitions):



Figure 2.1 - scripts/hw8_create_topics.sh: kafka-topics.sh --create for all topics (Figure 1.10 shows the result).

II. Planner, Writer and Reviewer code

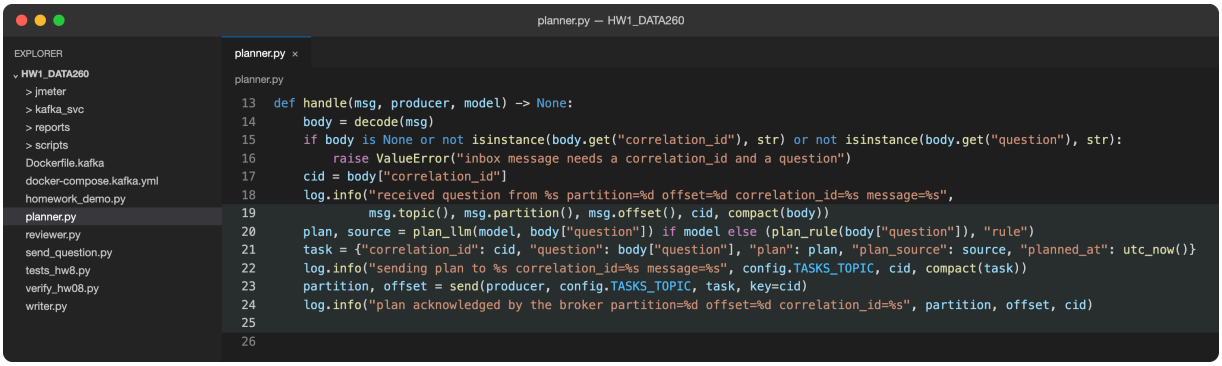


Figure 2.2 - planner.py: read the question from the inbox, write a plan to tasks (correlation_id kept, used as message key).

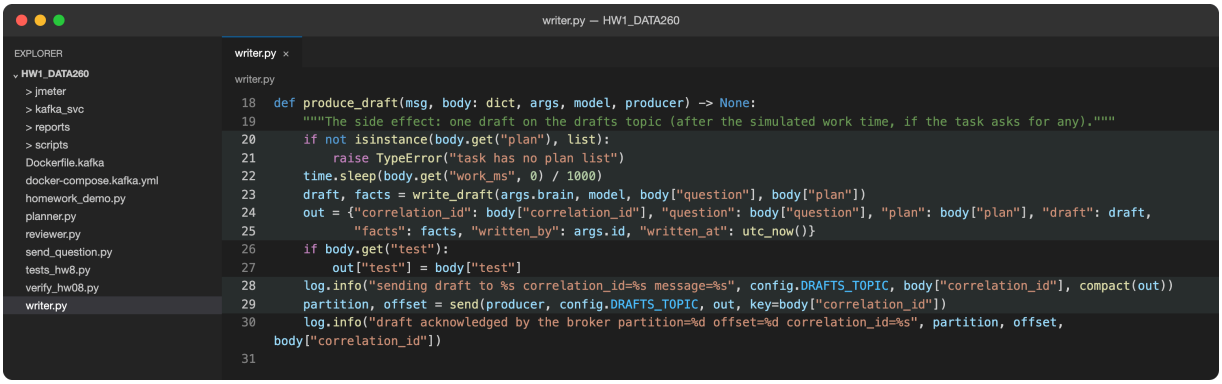


Figure 2.3 - writer.py: produce_draft() - facts from the tools, model draft, publish to drafts.

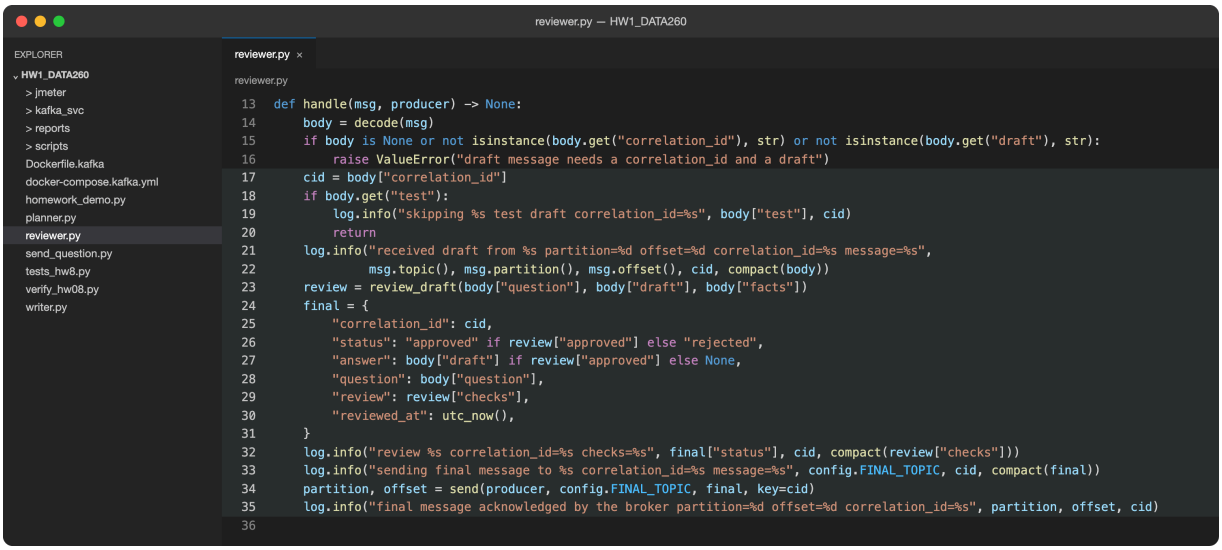


Figure 2.4 - reviewer.py: read a draft, run the checks, publish the final message.

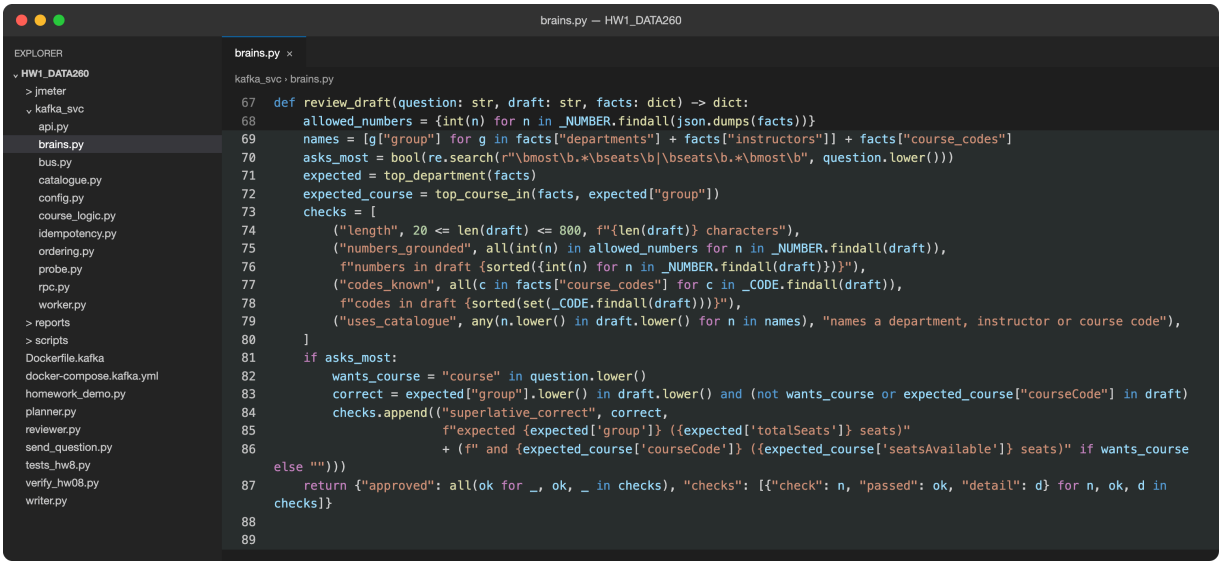


Figure 2.5 - kafka_svc/brains.py: review_draft() - the Reviewer's deterministic checks.

III. One complete message journey and the final JSON

Each agent ran in its own terminal (`python planner.py`, `python writer.py`, `python reviewer.py`, then `python send_question.py`). The figure shows the log lines of the four terminals for one correlation_id, one block per terminal; the correlation_id is the same in every stage.

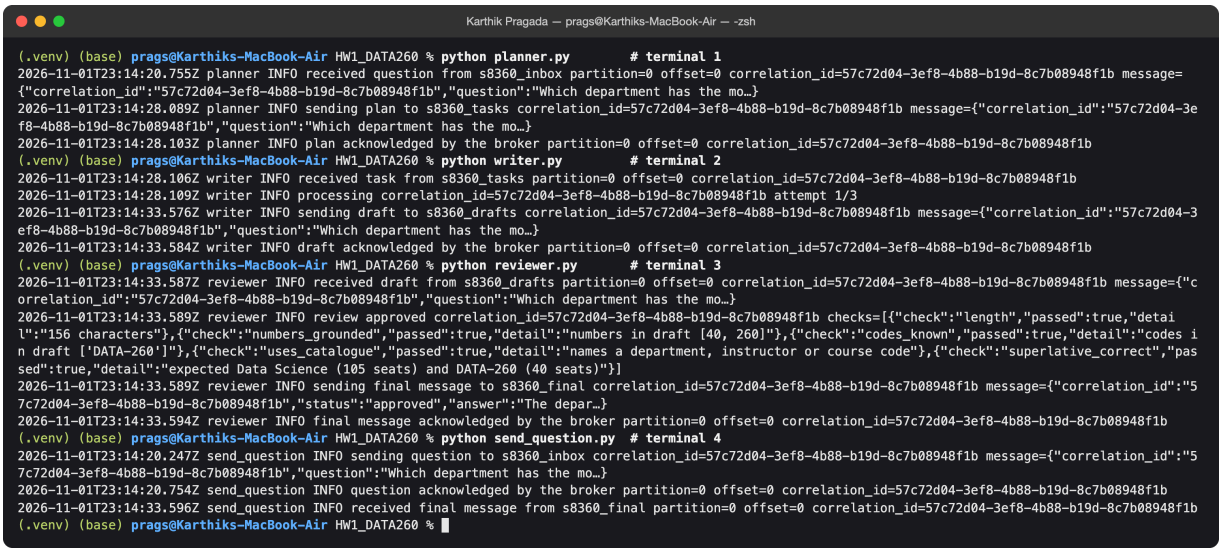


Figure 2.6 - Message journey inbox -> tasks -> drafts -> final (log lines of the four terminals, same correlation_id; payloads shortened with ...).

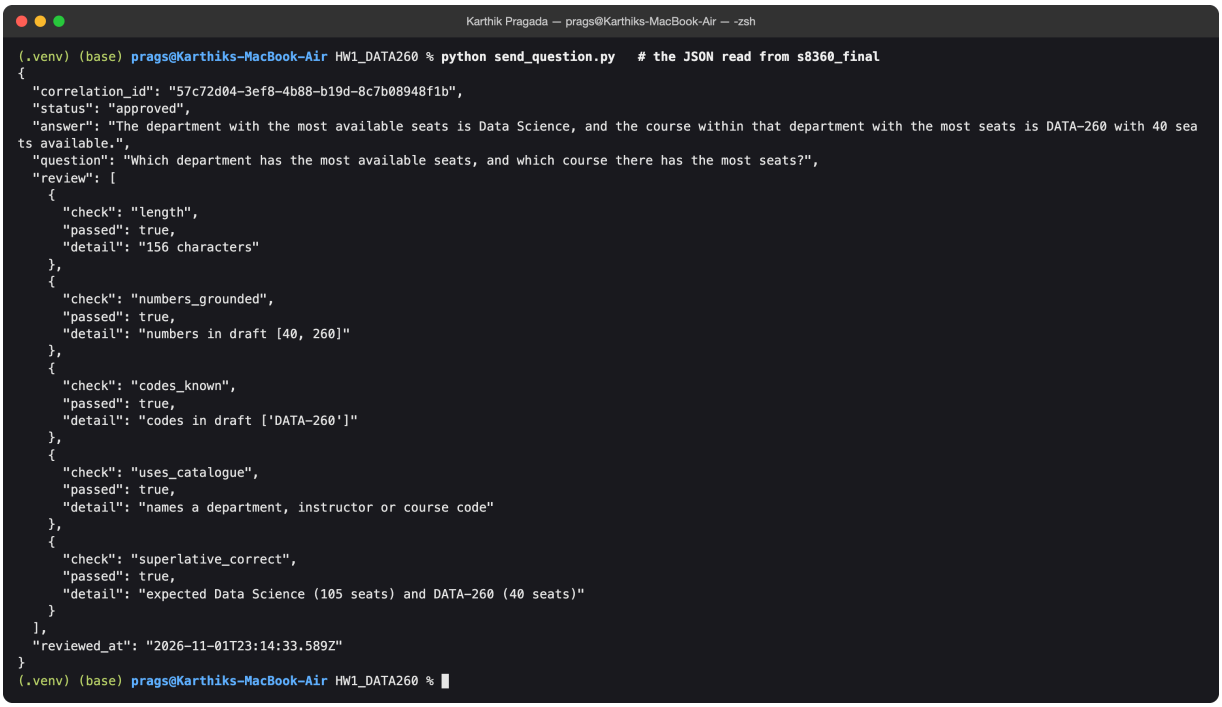


Figure 2.7 - The final JSON read from s8360_final: status approved and the answer, with the Reviewer's checks.

IV. How Kafka helped the agents coordinate

Stage	Seconds
question sent -> Planner receives	0.51
Planner (qwen3:8b plan)	7.33
tasks topic -> Writer receives	0.02
Writer (facts + qwen3:8b draft)	5.47
drafts topic -> Reviewer receives	0.01
Reviewer (deterministic checks)	0.0
final topic -> client receives	0.01
end to end	13.35

Kafka was the only coupling between the agents. Each stage could be started, stopped and restarted on its own, and a message waited on its topic until the next agent was ready (the Planner and Writer start-up order does not matter). The correlation_id travels inside every message, so the final answer can be matched to the question that caused it even with several questions in flight, and each topic keeps a replayable record of what every agent received and produced. The two model calls (7.3 s and 5.5 s) account for almost the whole 13.4 s; after the first message Kafka itself added about 10-20 ms per hop.

The Reviewer is also a real quality gate. A second question ("How many courses does Dr. Priya Nair teach and how many seats do they offer in total?") produced a draft with 75 seats; the catalogue says 70 (40 + 30). The Reviewer rejected it (status `rejected`, answer `null`), so no wrong number reached the user.

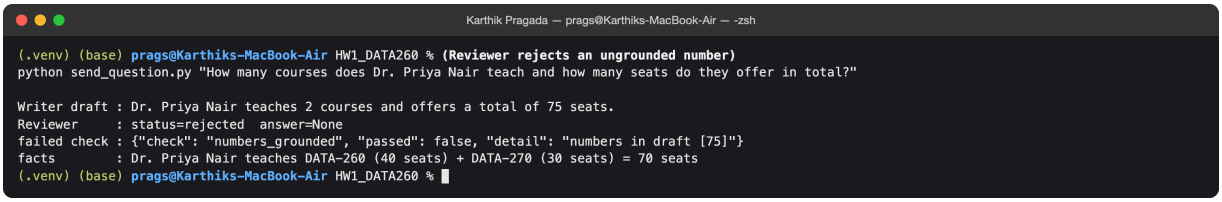


Figure 2.8 - A draft with an ungrounded number is rejected by the numbers_grounded check.

Part 3 - Failure Behavior

All four tests run the real Writer as separate OS processes against the same broker (`scripts/hw8_failure_tests.py`). The "before" run switches one fix off with a Writer flag (`--no-idempotency`, `--auto-commit`, `--no-dlq`); the "after" run is the default Writer. Every test uses its own consumer group so earlier messages cannot interfere. Offsets are read from the broker with `kafka-consumer-groups.sh --describe`, not from the Writer's own log.

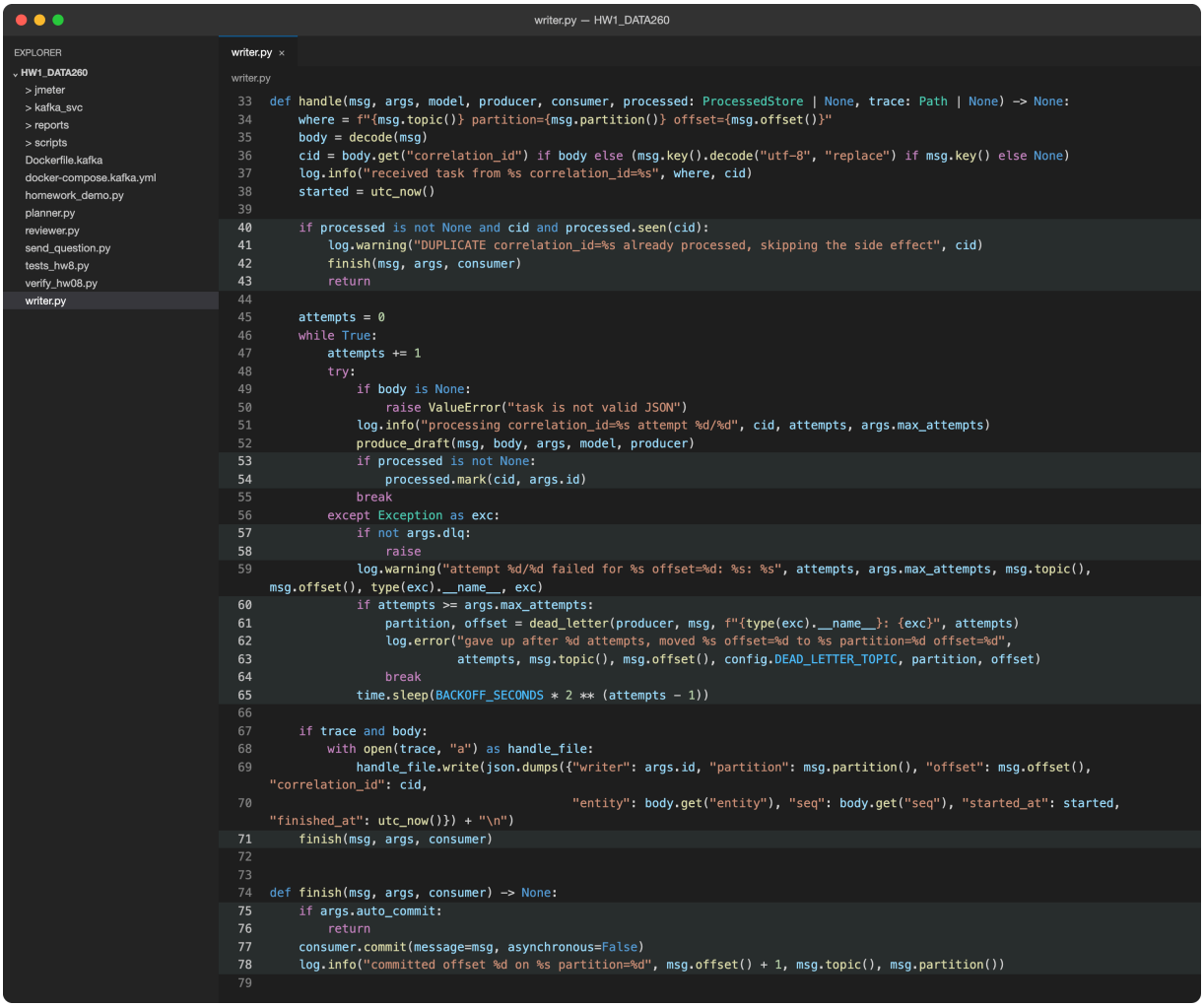


Figure 3.1 - writer.py handle(): idempotency check (lines 40-43), draft + mark as processed (53-54), bounded retries and dead-letter (57-65) and the commit after processing (75-78).

I. Duplicate delivery and idempotency

The same task (same correlation_id, same bytes) is delivered twice, as happens when a message is redelivered or a producer retries. Without protection the side effect (the draft on the drafts topic) happens twice. The fix records every processed correlation_id in a small SQLite table that survives a Writer restart and skips a replay. (Limit: if the Writer crashed between publishing the draft and recording the id, one duplicate is still possible - delivery stays at-least-once.)

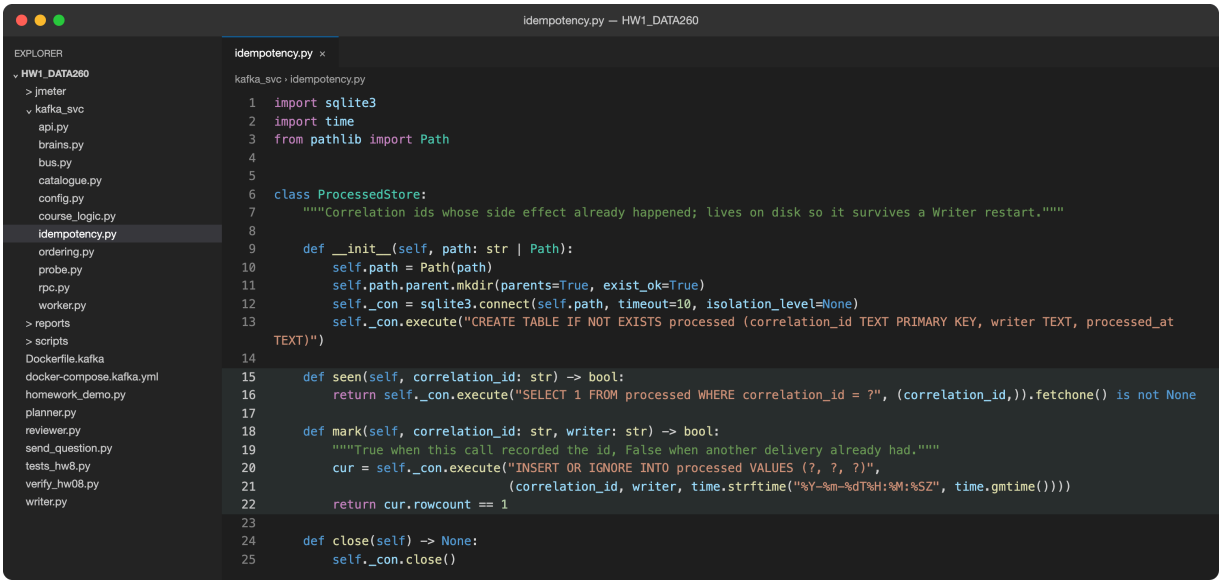


Figure 3.2 - kafka_svc/idempotency.py: the processed-ids store (INSERT OR IGNORE makes the check-and-record atomic).

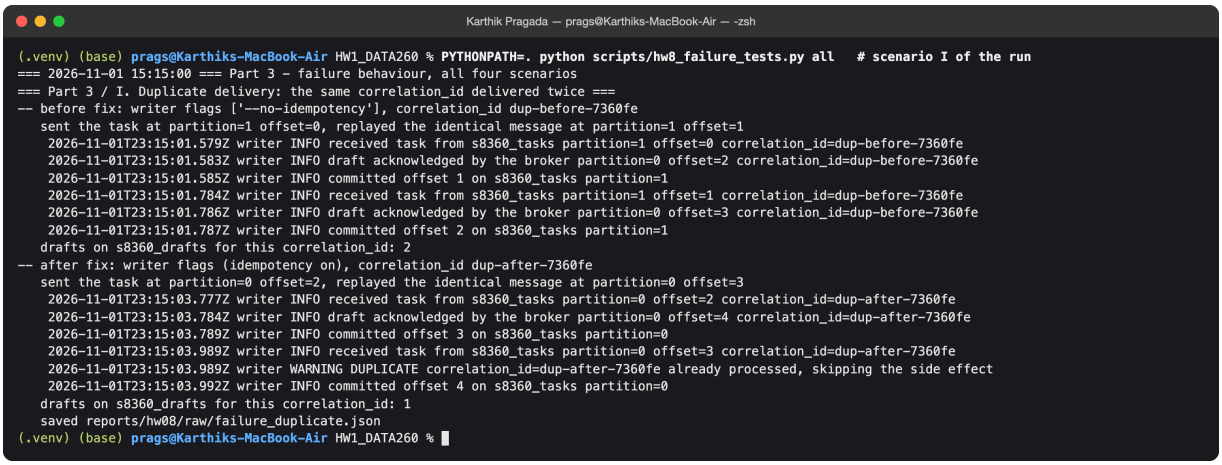


Figure 3.3 - Duplicate replay: 2 drafts without idempotency, 1 draft with it (the replay is logged as DUPLICATE and skipped).

II. Consumer death and offset commit

The Writer is killed with SIGKILL while it is busy with an 8-second task and then restarted. With the client's automatic commit, the offset is committed in the background before the work is finished, so after the restart the broker thinks the message is done and it is lost. The fix commits the offset only after the draft was published and acknowledged; the message then stays uncommitted (lag 1) until the work is really finished.

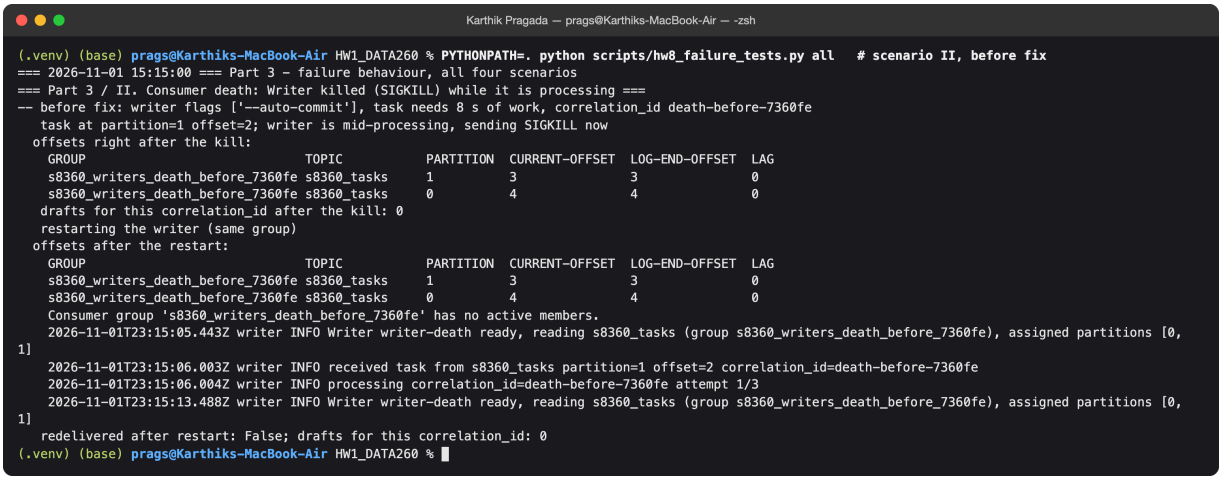


Figure 3.4 - Before: lag 0 after the kill; the second "ready" line is the restarted Writer and nothing is received after it - 0 drafts, the task is lost.

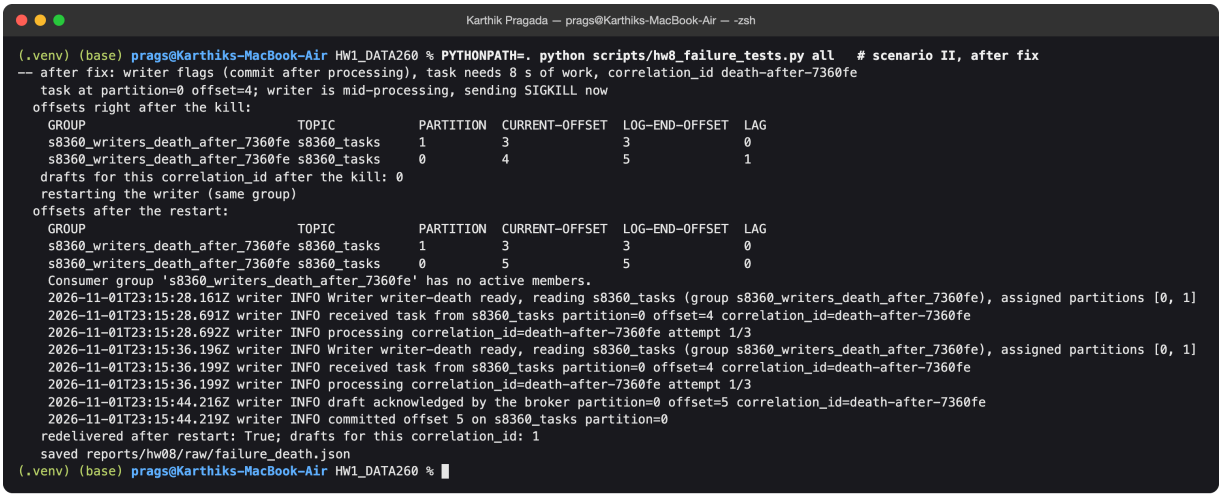


Figure 3.5 - After: lag 1 after the kill; the second "ready" line is the restarted Writer, which receives the same task (partition 0, offset 4) again - 1 draft, offset committed only after it.

III. Poison message, bounded retries and dead-letter topic

A truncated (invalid) JSON task is followed by a good task on the same partition. Without a fix the Writer crashes on the bad message, and because the offset never advances it crashes on it again after every restart, so the good task behind it never runs. With the fix the message is retried up to 3 times (0.5 s and 1 s back-off), then written to `s8360_dead_letter` with its source topic/partition/offset, the error and the original bytes; the offset is committed and the next message is processed.

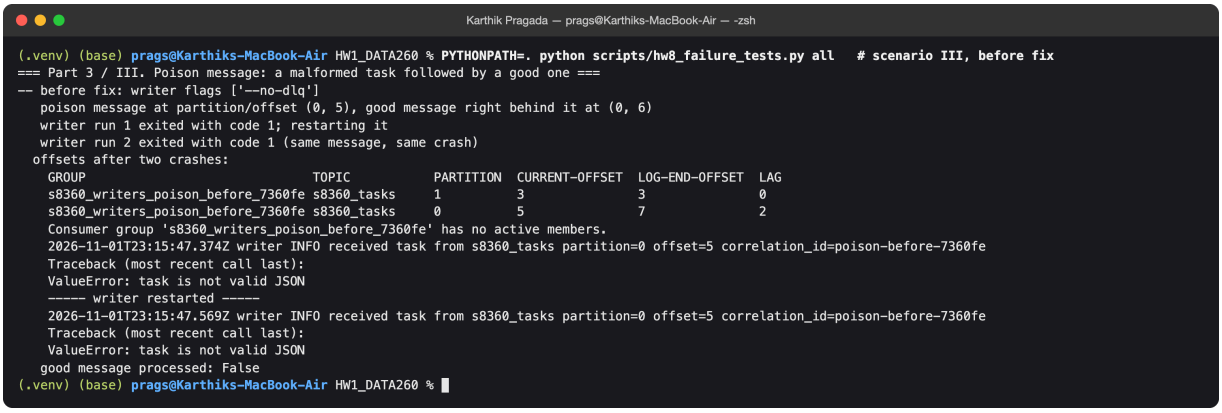


Figure 3.6 - Before: the Writer exits with code 1 on the poison message, again after the restart; lag stays at 2 and the later message is never processed.

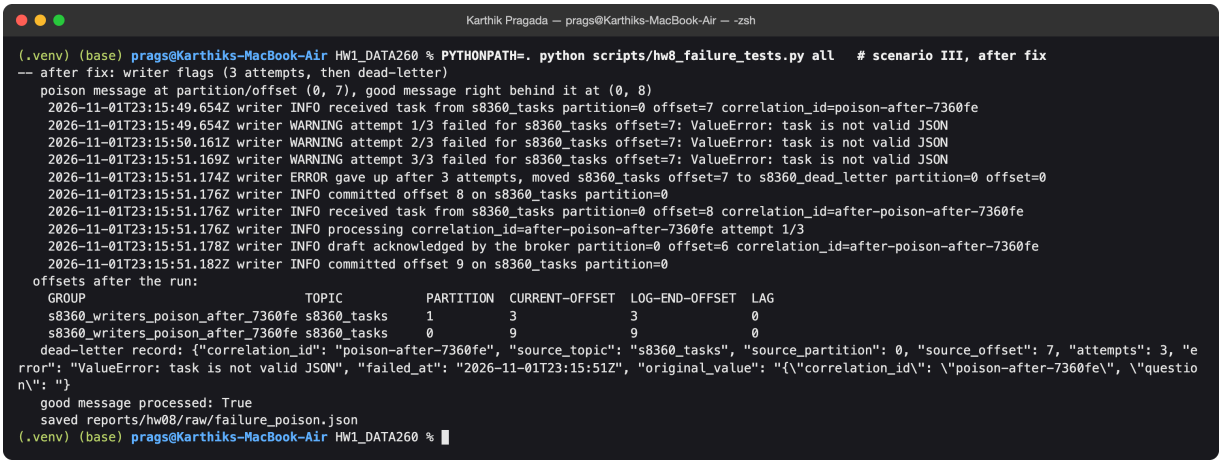


Figure 3.7 - After: 3 attempts, dead-letter, commit, then the good message is processed (lag 0).

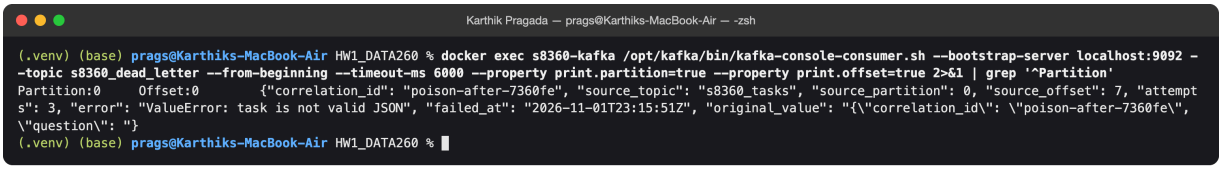


Figure 3.8 - The dead-letter topic read with the Kafka console consumer.

IV. Ordering with two partitions and two consumers

20 related messages are generated from VERIFY_SEED 268360 (several updates per course, numbered per course by `seq`, with a seeded 120-400 ms work time) and sent through `s8360_tasks` (2 partitions) to two Writers in the same group. Without a key the producer spreads each course's messages over both partitions, so two Writers process them at the same time and out of order. With the course code as message key all messages of a course go to one partition, which one Writer reads in order. The results of this run are random for the unkeyed case; keyed results are deterministic.

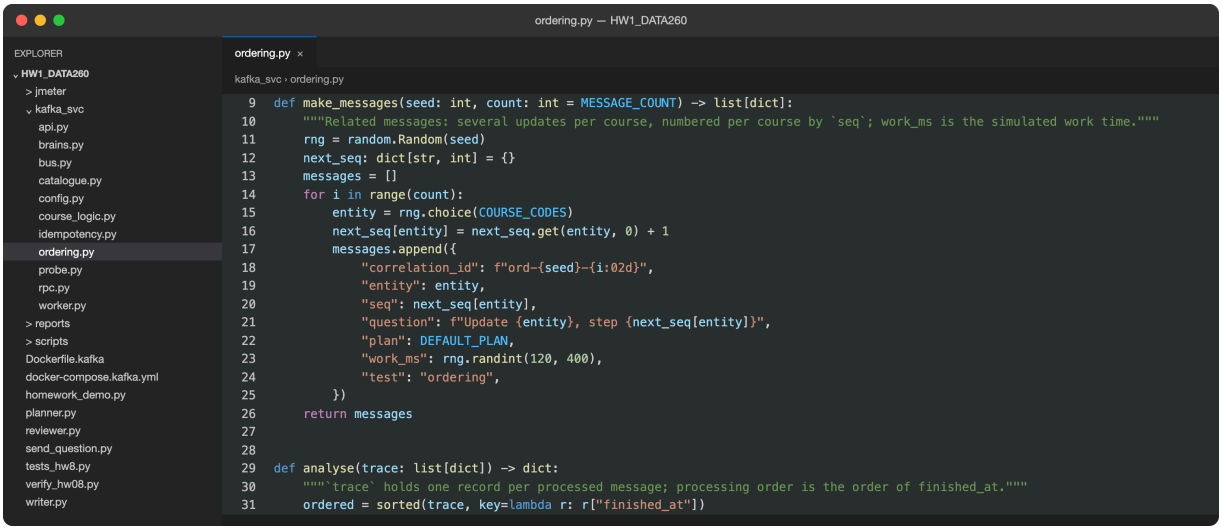


Figure 3.9 - kafka_svc/ordering.py: the seeded related messages.

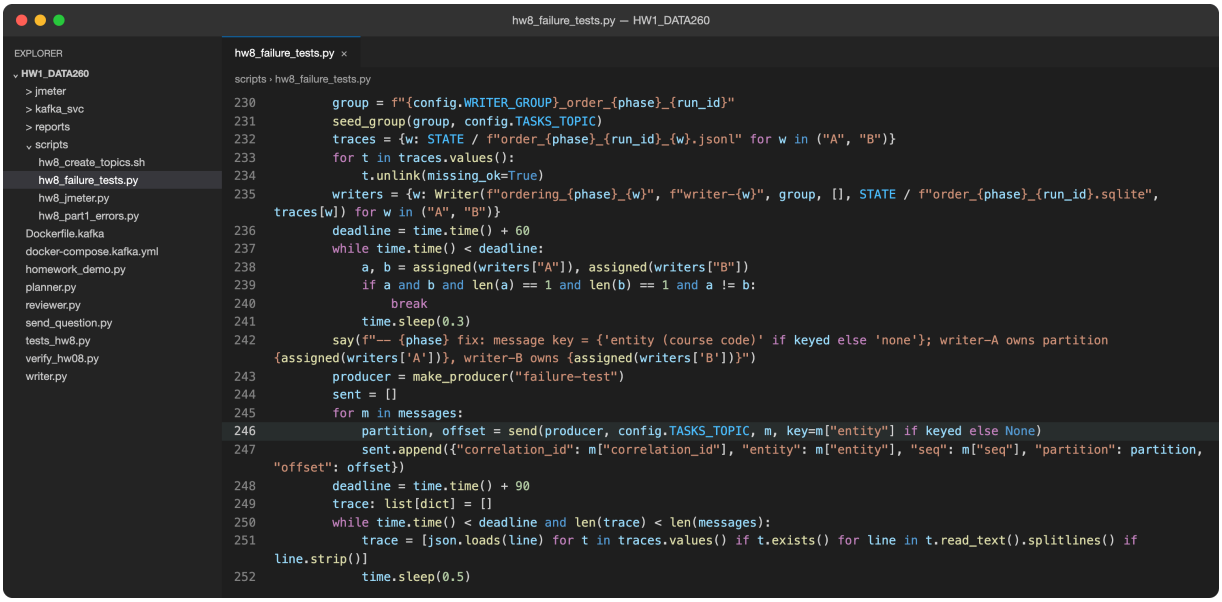


Figure 3.10 - scripts/hw8_failure_tests.py: the only difference between the two runs is the message key (line 246).

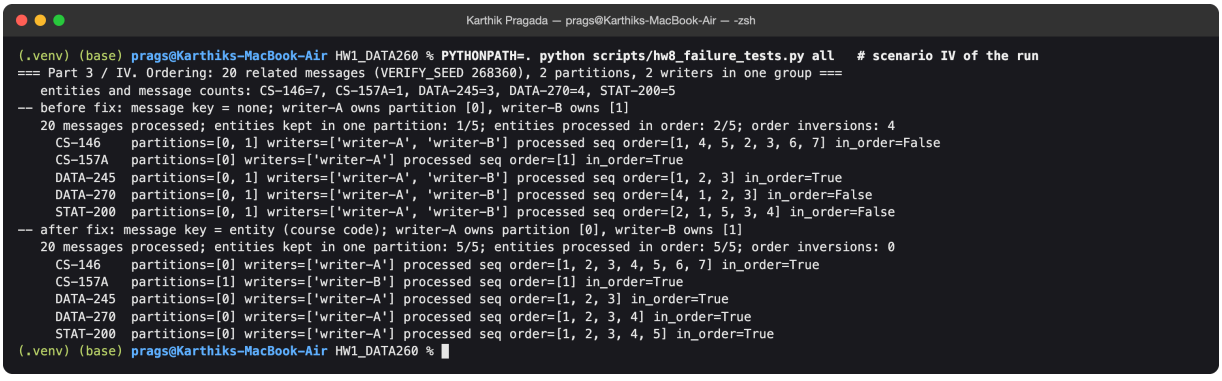


Figure 3.11 - Processing order per course without and with the key (seq numbers in the order they were processed).

Entity	Msgs	No key: partitions	No key: processed seq order	Key: partitions	Key: processed seq order
CS-146	7	[0, 1]	[1, 4, 5, 2, 3, 6, 7]	[0]	[1, 2, 3, 4, 5, 6, 7]
CS-157A	1	[0]	[1]	[1]	[1]
DATA-245	3	[0, 1]	[1, 2, 3]	[0]	[1, 2, 3]
DATA-270	4	[0, 1]	[4, 1, 2, 3]	[0]	[1, 2, 3, 4]
STAT-200	5	[0, 1]	[2, 1, 5, 3, 4]	[0]	[1, 2, 3, 4, 5]

Without a key: 4 order inversions, 2 of 5 courses in order, 1 of 5 in one partition. With the key: 0 inversions, 5 of 5 courses in order, all in one partition. Trade-off: with this seed four of the five courses hash to partition 0, so one Writer does most of the work; ordering per course costs some balance.

V. Comparison with the HW2 in-process graph and the failure table

Two benefits gained by moving to Kafka. (1) Independent scaling and deployment: in HW2 the Planner and Reviewer were nodes of one LangGraph graph (`agent_graph.py`) inside one process; now each agent is its own process, and a slow Writer can be scaled by starting a second instance in the same consumer group (Part 3 IV runs two). (2) Decoupled failure: when the Writer was killed the Planner and Reviewer kept running and the task waited on the topic; in HW2 one exception or one crash stopped the whole graph. A side benefit is the persistent, replayable log of every message.

Two responsibilities I now have to handle myself. (1) Duplicate delivery: a function call happens exactly once, a message can arrive more than once, so the Writer needs idempotency keyed by correlation_id and an explicit rule for when the offset is committed. (2) Ordering and bad messages: order is only guaranteed inside one partition, so related messages need a key, and a message that can never succeed needs a retry limit and a dead-letter topic; a function call in HW2 simply raised an exception to its caller.

Failure scenario	Before fix	After fix
Duplicate delivery	same correlation_id delivered twice -> **2 drafts** (side effect twice). Log: Figure 3.3	**1 draft**; second delivery logged as DUPLICATE and skipped. Log: Figure 3.3
Consumer killed mid-work	offset auto-committed early: lag after kill **0**, redelivered after restart: **False**, drafts **0** (message lost). Log: Figure 3.4	offset committed after the draft: lag after kill **1**, redelivered: **True**, drafts **1**. Log: Figure 3.5
Poison message	Writer exits with code 1, again after restart; later message processed: **False**, lag 2. Log: Figure 3.6	**3 attempts**, moved to s8360_dead_letter (offset 7); later message processed: **True**, lag 0. Log: Figures 3.7-3.8
Two partitions / two consumers	no key: **4 inversions**, 2/5 courses in order, 1/5 in one partition. Log: Figure 3.11	key = course code: **0 inversions**, 5/5 in order, 5/5 in one partition. Log: Figure 3.11

Self-check: offline tests and verification.json

`tests_hw8.py` (24 tests; no Kafka, network or model; the Writer's retry/commit logic is tested with a fake consumer) and `verify_hw08.py`, a smoke test on the live system that writes `reports/hw08/verification.json` (containers and topics, CREATE_COURSE -> GET_COURSE through Kafka and through FastAPI, a live Planner -> Writer -> Reviewer run, the saved Part 3 data, Ruff and the deliverables). It creates only temporary consumer groups and files and never modifies application code.

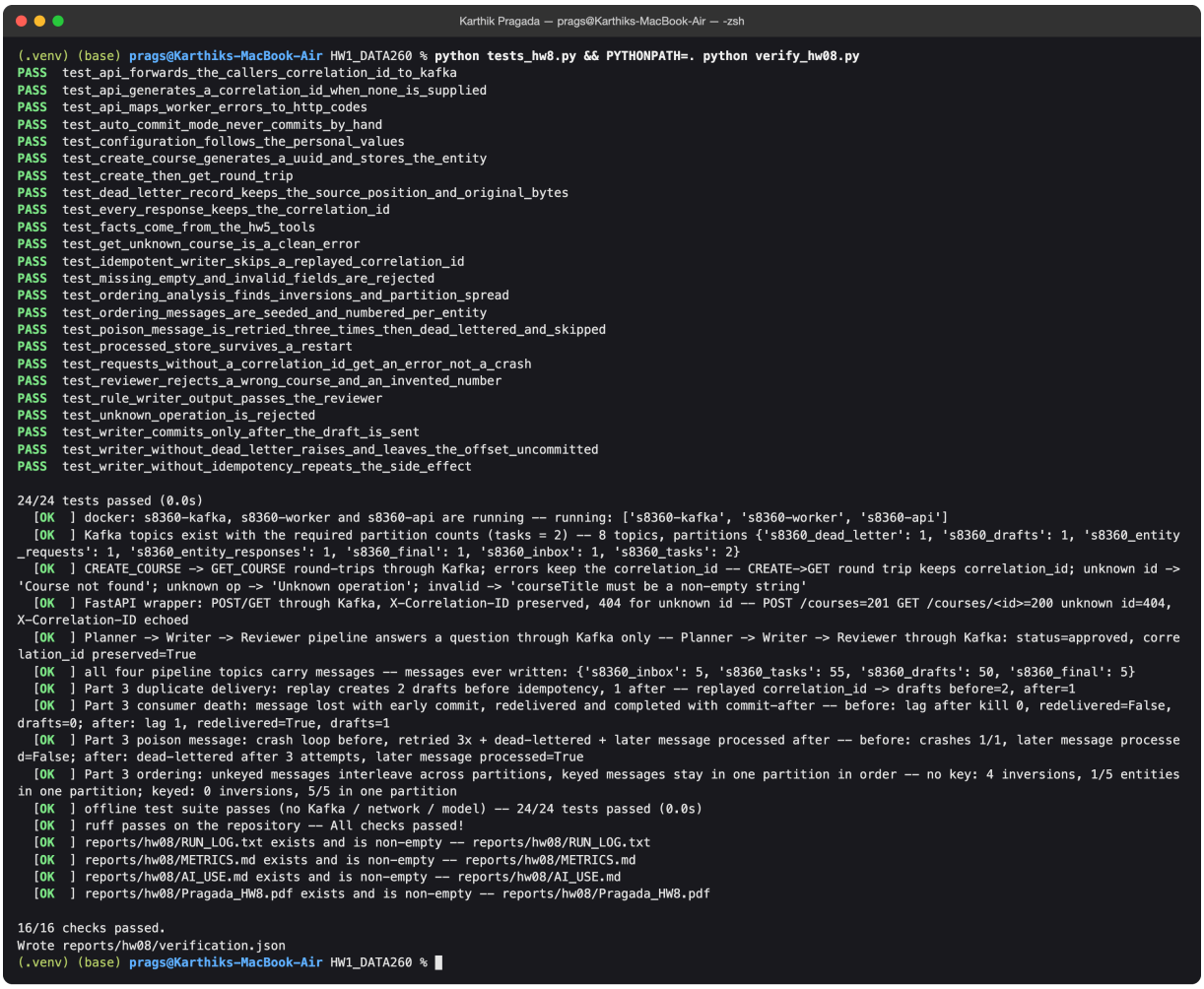


Figure 4.1 - tests_hw8.py followed by verify_hw08.py (16/16 checks).

AI Use

1. What did you use an AI assistant for, and what did you do yourself?

I extended my existing HW1–HW7 repository for Homework 8 instead of creating a separate copy, and I kept everything unpushed until I could review the changes. I gave Claude Code the HW8 assignment PDF, and I kept the report format and the "do not push until I review" rule from the earlier homeworks.

I used Claude Code to write and run the new code: the kafka_svc/ package (request/response client, worker, FastAPI wrapper), the Docker setup for Kafka, worker and API, homework_demo.py, the Planner, Writer, Reviewer and send_question.py, the failure-test harness, tests_hw8.py, verify_hw08.py, and the report generator. My own part was to review the code and outputs, check that the numbers in the report match the raw files, and decide what is submitted and pushed.

2. One AI-produced output that was wrong/unsuitable, or one thing I independently verified

The first end-to-end run of the three-agent pipeline produced a wrong answer that the Reviewer approved. The question was "Which department has the most available seats, and which course there has the most seats?". The Writer said Data Science and DATA-245, but DATA-245 has 35 seats and DATA-260 has 40. The Writer had only been given department totals, so the model guessed the course, and the Reviewer only checked that the course code existed in the catalogue.

The first version of my Part 3 harness also reported a wrong result: in the "consumer killed" test it said the message was redelivered after the restart for the unfixed Writer, even though no draft was ever produced. It had matched the log line from the first run, not from the restarted one.

3. How did you detect the problem or verify the result?

For the wrong answer, I compared the final JSON with the catalogue numbers in the facts the Writer received, which showed that the per-course seat counts were never in the prompt. I also saw that the same answer came back for three different questions, which made me check the running processes: an older copy of the Reviewer from before my fix was still running in the same consumer group and approving drafts (my pkill pattern used a lowercase "python", while macOS shows "Python").

For the harness bug, the output contradicted itself: "redelivered: True" next to "drafts: 0" and a consumer-group lag of 0. I did not trust the Writer's own log for the consumer-death test; I used the broker's view (kafka-consumer-groups.sh --describe) before and after the kill and counted the drafts that actually reached the drafts topic.

4. What did you change and why does it work now?

The Writer now receives the full course list (found with the HW5 search_courses and course_stats tools) and the Reviewer has a superlative_correct check that recomputes the top department and its top course from the same facts. With that, the correct answer (Data Science, DATA-260, 40 seats) is approved, and a wrong or invented answer is rejected: a second question about Dr. Nair's seats came back "rejected" because the model wrote 75 seats and the facts say 70. I also stopped all old agent processes by pid before the final runs.

I fixed the harness to require the second occurrence of the "received task" line after the restart. The final numbers agree with the broker: with early auto-commit the group lag was 0 after the kill and the message was lost; with commit-after-processing the lag was 1, the message was redelivered, and exactly one draft was produced.

Limitations I want to state plainly: the broker is a single node with replication factor 1; at-least-once delivery plus the idempotency check gives effectively-once results only if the Writer does not crash between sending the draft and recording the correlation_id (that window is

small but not zero); the unkeyed ordering result is random, so the exact inversion count changes from run to run; and keying by course code puts most of this seeded data on one partition, so one Writer is busier than the other.